

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH  
TRƯỜNG ĐẠI HỌC BÁCH KHOA  
KHOA KHOA HỌC KỸ THUẬT MÁY TÍNH



HỌC MÁY - CO3117

---

Phân tích cảm xúc  
Sentiment Analysis

---

|             |                        |         |
|-------------|------------------------|---------|
| Giảng viên: | T.s Trương Vĩnh Lân    |         |
| Sinh viên:  | Trần Nguyễn Mạnh Cường | 2210446 |
|             | Nguyễn Trọng Nghĩa     | 2312271 |
|             | Trần Trung Kiên        | 2311744 |
|             | Nguyễn Đăng Khoa       | 2211617 |
|             | Nguyễn Quang Bảo       | 2410276 |

## Table of contents

|                                                                                                   |           |
|---------------------------------------------------------------------------------------------------|-----------|
| <b>1 Project Archive:</b>                                                                         | <b>4</b>  |
| <b>2 Mô tả bài toán phân tích cảm xúc &amp; mục tiêu</b>                                          | <b>4</b>  |
| 2.1 Khái niệm . . . . .                                                                           | 4         |
| 2.2 Tại sao cần Phân tích cảm xúc? . . . . .                                                      | 4         |
| 2.3 Ứng dụng thực tiễn trên thị trường . . . . .                                                  | 4         |
| 2.4 Mục tiêu nghiên cứu . . . . .                                                                 | 4         |
| <b>3 Cơ sở lý thuyết</b>                                                                          | <b>5</b>  |
| 3.1 Định nghĩa và Lựa chọn Dữ liệu (Data Definition & Selection) . . . . .                        | 5         |
| 3.1.1 Định nghĩa Dữ liệu . . . . .                                                                | 5         |
| 3.1.2 Lý do lựa chọn Dữ liệu . . . . .                                                            | 5         |
| 3.2 Phân tích Dữ liệu Khám phá (Exploratory Data Analysis - EDA) . . . . .                        | 5         |
| 3.2.1 Phân phối chiều dài văn bản . . . . .                                                       | 5         |
| 3.2.2 Tần suất xuất hiện của từ (Word Frequency) . . . . .                                        | 5         |
| 3.3 Tiền xử lý và Làm sạch Dữ liệu (Data Cleaning & Preprocessing) . . . . .                      | 5         |
| 3.4 Cơ sở Toán học của việc Trích xuất Đặc trưng (Feature Extraction) . . . . .                   | 6         |
| 3.4.1 Term Frequency (TF) . . . . .                                                               | 6         |
| 3.4.2 Inverse Document Frequency (IDF) . . . . .                                                  | 6         |
| 3.4.3 Trọng số TF-IDF . . . . .                                                                   | 6         |
| 3.4.4 Word2Vec . . . . .                                                                          | 6         |
| 3.4.5 GloVe (Global Vectors for Word Representation) . . . . .                                    | 6         |
| 3.4.6 So sánh Word Embeddings và TF-IDF . . . . .                                                 | 6         |
| 3.5 Mô hình học máy truyền thống . . . . .                                                        | 6         |
| 3.5.1 Hồi quy Logistic (Logistic Regression) . . . . .                                            | 6         |
| 3.5.1.a Hàm giả thuyết (Hypothesis Function) . . . . .                                            | 7         |
| 3.5.1.b Hàm mất mát (Binary Cross-Entropy Loss) . . . . .                                         | 7         |
| 3.5.2 Support Vector Machine - SVM . . . . .                                                      | 7         |
| 3.5.2.a Hàm mất mát Hinge (Hinge Loss) . . . . .                                                  | 8         |
| 3.5.3 Bộ phân loại Naive Bayes Đa thức (Multinomial Naive Bayes) . . . . .                        | 8         |
| 3.6 Mô hình học sâu . . . . .                                                                     | 8         |
| 3.6.1 Long Short-Term Memory (LSTM) . . . . .                                                     | 8         |
| 3.6.1.a Tổng quan . . . . .                                                                       | 8         |
| 3.6.1.b Cấu trúc toán học . . . . .                                                               | 9         |
| 3.6.2 BERT (Bidirectional Encoder Representations from Transformers) . . . . .                    | 9         |
| 3.6.2.a Cơ chế Attention . . . . .                                                                | 9         |
| 3.6.2.b Kiến trúc BERT . . . . .                                                                  | 10        |
| 3.6.2.c Biểu diễn đầu vào . . . . .                                                               | 10        |
| 3.6.2.d Pre-training . . . . .                                                                    | 10        |
| 3.6.2.e Fine-tuning cho tác vụ cụ thể . . . . .                                                   | 10        |
| <b>4 Phương pháp Đánh giá Mô hình</b>                                                             | <b>10</b> |
| 4.1 Ma trận nhầm lẫn (Confusion Matrix) . . . . .                                                 | 11        |
| <b>5 Hiện thực bài toán</b>                                                                       | <b>12</b> |
| 5.1 Tiền xử lý dữ liệu . . . . .                                                                  | 12        |
| 5.2 Xử lý Dữ liệu Thưa và Trích xuất Đặc trưng (Sparsity Handling & Feature Extraction) . . . . . | 12        |
| 5.3 Cấu hình Mô hình và Điều chuẩn (Model Configuration & Regularization) . . . . .               | 12        |
| 5.4 Tối ưu hóa Siêu tham số (Hyperparameter Tuning) . . . . .                                     | 15        |
| 5.4.1 Thiết lập Không gian tìm kiếm (Search Space) . . . . .                                      | 15        |
| 5.4.2 Kiểm chứng chéo K-Fold (K-Fold Cross-Validation) . . . . .                                  | 15        |
| 5.4.3 Kết quả Tối ưu hóa . . . . .                                                                | 15        |
| 5.5 Chiến lược Huấn luyện và Đánh giá (Training & Evaluation Framework) . . . . .                 | 16        |
| 5.5.1 Đối với mô hình truyền thống: . . . . .                                                     | 16        |
| 5.5.2 Đối với mô hình học sâu (LSTM): . . . . .                                                   | 16        |
| 5.5.3 Đối với mô hình học sâu (BERT): . . . . .                                                   | 16        |
| 5.5.4 Sự khác biệt trong chiến thuật huấn luyện . . . . .                                         | 16        |

|          |                                                                                      |           |
|----------|--------------------------------------------------------------------------------------|-----------|
| 5.5.5    | Điều chuẩn L2 (L2 Regularization) . . . . .                                          | 16        |
| <b>6</b> | <b>Kết quả và đánh giá</b>                                                           | <b>18</b> |
| 6.1      | Phân tích đặc điểm dữ liệu (Exploratory Data Analysis - EDA) . . . . .               | 18        |
| 6.1.1    | Phân phối nhãn dữ liệu (Class Distribution) . . . . .                                | 18        |
| 6.1.2    | Phân phối độ dài văn bản (Word Count Distribution) . . . . .                         | 18        |
| 6.1.3    | Tần suất xuất hiện của từ (Word Frequency & Word Cloud) . . . . .                    | 19        |
| 6.2      | Mô tả Pipeline và các tham số đã thử nghiệm . . . . .                                | 20        |
| 6.2.1    | Tổng quan kiến trúc Pipeline . . . . .                                               | 20        |
| 6.2.2    | Bảng tổng hợp tham số cấu hình (Hyperparameter Configuration) . . . . .              | 21        |
| 6.2.3    | Tóm tắt thực nghiệm Grid Search . . . . .                                            | 22        |
| 6.3      | Các chỉ số đánh giá (Precision, Recall và F1-Score) . . . . .                        | 23        |
| 6.4      | Pipeline truyền thống: . . . . .                                                     | 24        |
| 6.4.1    | Logistic Regression: . . . . .                                                       | 24        |
| 6.4.2    | Naives Bayes: . . . . .                                                              | 24        |
| 6.4.3    | SVM: . . . . .                                                                       | 25        |
| 6.4.4    | So sánh giữa các model truyền thống: . . . . .                                       | 25        |
| 6.5      | Deeplearning Pipeline: . . . . .                                                     | 25        |
| 6.6      | So sánh giữa pipeline truyền thống, deeplearning và model hiện đại - BERT: . . . . . | 26        |
| 6.6.1    | Ràng buộc về Kích thước Dữ liệu (Data Starvation) . . . . .                          | 26        |
| 6.6.2    | Bản chất của Bài toán Phân tích Cảm xúc . . . . .                                    | 27        |
| 6.6.3    | Chênh lệch trong Không gian Tối ưu hóa (Hyperparameter Tuning) . . . . .             | 27        |

## Bảng phân công công việc

| Họ và tên              | MSSV    | Công việc                                     | Hoàn thành | Đóng góp |
|------------------------|---------|-----------------------------------------------|------------|----------|
| Trần Nguyễn Mạnh Cường | 2210446 | Tìm hiểu hiện thực đề tài, viết báo cáo LaTeX | 100%       | 35%      |
| Nguyễn Quang Bảo       | 2410276 | Tìm hiểu và hiện thực BERT                    | 100%       | 35%      |
| Nguyễn Trọng Nghĩa     | 2312271 | Tìm hiểu đề tài                               | 100%       | 10%      |
| Trần Trung Kiên        | 2311744 | Tìm hiểu đề tài                               | 100%       | 10%      |
| Nguyễn Đăng Khoa       | 2211617 | Tìm hiểu đề tài                               | 100%       | 10%      |

## 1 Project Archive:

Github: <https://github.com/hufzohel/Sentiment-Analysis>

Google Colab: [https://colab.research.google.com/drive/1iLm98k\\_Gi0H5u7A01TJJ\\_i\\_SXxkZXfUe?usp=sharing](https://colab.research.google.com/drive/1iLm98k_Gi0H5u7A01TJJ_i_SXxkZXfUe?usp=sharing)

## 2 Mô tả bài toán phân tích cảm xúc & mục tiêu

### 2.1 Khái niệm

Phân tích cảm xúc (Sentiment Analysis) là một nhánh của Xử lý ngôn ngữ tự nhiên (NLP) tập trung vào việc xác định và trích xuất các trạng thái cảm xúc, thái độ hoặc ý kiến chủ quan từ văn bản. Mục tiêu chính là phân loại một đơn vị văn bản (câu, đoạn văn hoặc toàn bộ tài liệu) vào các nhóm cảm xúc cụ thể, phổ biến nhất là: Tích cực (Positive), Tiêu cực (Negative) hoặc Trung tính (Neutral).

### 2.2 Tại sao cần Phân tích cảm xúc?

Trong kỷ nguyên bùng nổ dữ liệu số, việc hiểu được tâm lý con người thông qua văn bản mang lại những giá trị cốt lõi sau:

- **Xử lý dữ liệu quy mô lớn:** Con người không thể đọc và phân loại hàng triệu bình luận trên mạng xã hội mỗi ngày. Machine Learning cho phép tự động hóa quy trình này với tốc độ vượt trội.
- **Tính khách quan:** Các mô hình được huấn luyện tốt giúp giảm thiểu sự thiên kiến cá nhân khi đánh giá ý kiến, mang lại cái nhìn nhất quán hơn về dữ liệu.
- **Phản ứng thời gian thực:** Doanh nghiệp có thể phát hiện ngay lập tức các khủng hoảng truyền thông hoặc sự không hài lòng của khách hàng để đưa ra giải pháp kịp thời.

### 2.3 Ứng dụng thực tiễn trên thị trường

Hiện nay, Phân tích cảm xúc đang được các doanh nghiệp và tổ chức ứng dụng rộng rãi trong nhiều lĩnh vực:

1. **Thương mại điện tử:** Tóm tắt các đánh giá (reviews) của khách hàng trên các sàn như Shopee, Tiki, Amazon giúp người mua sau dễ dàng ra quyết định và giúp người bán cải thiện chất lượng dịch vụ.
2. **Nghiên cứu thị trường và Chính trị:** Phân tích xu hướng ủng hộ của công chúng đối với một chính sách, một ứng cử viên hoặc để dự đoán biến động của thị trường chứng khoán dựa trên tin tức tài chính.

### 2.4 Mục tiêu nghiên cứu

Trong phạm vi môn học Machine Learning, nhóm thực hiện đề tài này với các mục tiêu cụ thể sau:

1. **Tìm hiểu quy trình tiền xử lý văn bản (Text Preprocessing):** Nghiên cứu cách chuẩn hóa dữ liệu văn bản như loại bỏ stop words, xử lý từ viết tắt, gán nhãn từ loại (POS tagging) và đặc biệt là kỹ thuật tách từ (Tokenization).
2. **Khám phá các phương pháp trích xuất đặc trưng (Feature Extraction):** Thực hành chuyển đổi văn bản sang dạng vector số học thông qua các kỹ thuật từ cơ bản đến chuyên sâu như Bag of Words (BoW), TF-IDF, hoặc các mô hình nhúng từ (Word Embeddings) như Word2Vec, FastText.
3. **Xây dựng và đánh giá mô hình Machine Learning:** Triển khai các thuật toán phân loại (Classification) để giải quyết bài toán Sentiment Analysis. Nhóm hướng tới việc so sánh hiệu năng giữa các mô hình truyền thống (như Naive Bayes, SVM) và các kiến trúc Deep Learning (như LSTM, GRU) hoặc Transformer (như BERT).
4. **Đánh giá độ chính xác thực tế:** Sử dụng các độ đo phổ biến như Accuracy, Precision, Recall và F1-score để đánh giá khả năng nhận diện cảm xúc của mô hình trên tập dữ liệu kiểm thử (Test set), từ đó hiểu rõ các thách thức như xử lý câu phủ định (Negation) hay sắc thái mỉa mai (Sarcasm).

### 3 Cơ sở lý thuyết

#### 3.1 Định nghĩa và Lựa chọn Dữ liệu (Data Definition & Selection)

##### 3.1.1 Định nghĩa Dữ liệu

Trong bài toán Phân tích Sắc thái (Sentiment Analysis), dữ liệu đầu vào  $\mathcal{D}$  là một tập hợp các văn bản (đánh giá, bình luận), được biểu diễn dưới dạng các cặp  $(x_i, y_i)$ , trong đó:

- $x_i \in \mathcal{X}$ : Là văn bản thứ  $i$  trong tập dữ liệu, bao gồm một chuỗi các ký tự hoặc từ vựng.
- $y_i \in \mathcal{Y}$ : Là nhãn cảm xúc tương ứng của  $x_i$ . Đối với bài toán phân loại nhị phân,  $\mathcal{Y} = \{0, 1\}$  (0 đại diện cho Tiêu cực - Negative, 1 đại diện cho Tích cực - Positive).

##### 3.1.2 Lý do lựa chọn Dữ liệu

Nhóm quyết định lựa chọn tập dữ liệu đánh giá văn bản (IMDB) dựa trên các tiêu chí sau:

- **Tính đại diện:** Ngôn ngữ tự nhiên trong các đánh giá chứa đựng nhiều yếu tố nhiễu (noise), từ lóng và sai ngữ pháp, phản ánh đúng thực tế của dữ liệu phi cấu trúc.
- **Độ cân bằng:** Giả sử tập dữ liệu có  $N$  mẫu, tập dữ liệu lý tưởng sẽ có tỷ lệ  $P(y = 1) \approx P(y = 0) \approx 0.5$ . Nếu dữ liệu mất cân bằng (Imbalanced Data), nhóm sẽ phải áp dụng các kỹ thuật bù trừ như thay đổi trọng số lớp (Class Weights) hoặc nội suy dữ liệu.

#### 3.2 Phân tích Dữ liệu Khám phá (Exploratory Data Analysis - EDA)

Quá trình EDA nhằm mục đích thấu hiểu phân phối thống kê của tập dữ liệu trước khi huấn luyện mô hình. Hai đại lượng thống kê quan trọng đối với dữ liệu văn bản bao gồm:

##### 3.2.1 Phân phối chiều dài văn bản

Chiều dài của một văn bản  $x_i$ , ký hiệu là  $L_i$ , được tính bằng số lượng từ (tokens) có trong văn bản đó. Chiều dài trung bình  $\mu$  và phương sai  $\sigma^2$  của toàn bộ tập dữ liệu  $N$  văn bản được xác định bởi:

$$\mu = \frac{1}{N} \sum_{i=1}^N L_i$$
$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (L_i - \mu)^2$$

Việc xác định  $\mu$  và  $\sigma^2$  giúp đưa ra quyết định cắt tỉa (truncation) hoặc đệm (padding) chiều dài câu khi áp dụng các mô hình Học sâu (Deep Learning) ở giai đoạn sau.

##### 3.2.2 Tần suất xuất hiện của từ (Word Frequency)

Để phân tích ngữ nghĩa tổng quan, nhóm xem xét tần suất xuất hiện của từng từ vựng  $t$  trong toàn bộ kho ngữ liệu (corpus)  $C$ . Tần suất này giúp xây dựng Word Cloud và phát hiện các "stop words" chưa được lọc bỏ.

#### 3.3 Tiền xử lý và Làm sạch Dữ liệu (Data Cleaning & Preprocessing)

Dữ liệu văn bản thô thường ở dạng phi cấu trúc và chứa nhiều thông tin không đóng góp vào việc phân loại cảm xúc. Quá trình tiền xử lý được định nghĩa qua hàm biến đổi  $f: \mathcal{X} \rightarrow \mathcal{X}'$ , bao gồm các bước:

1. **Loại bỏ nhiễu (Noise Removal):** Sử dụng Biểu thức chính quy (Regular Expression - Regex) để loại bỏ các thẻ HTML, URL và các ký tự đặc biệt không mang ý nghĩa ngữ nghĩa.
2. **Chuẩn hóa (Normalization):** Chuyển đổi toàn bộ văn bản về chữ in thường (lowercasing) để đồng nhất không gian biểu diễn từ vựng, tức là  $Word(w) = Word(W)$ .
3. **Loại bỏ từ dừng (Stopword Removal):** Các từ xuất hiện với tần suất quá cao (ví dụ: "the", "a", "is" trong tiếng Anh) nhưng mang ít giá trị phân loại sắc thái sẽ bị loại bỏ khỏi không gian đặc trưng.

### 3.4 Cơ sở Toán học của việc Trích xuất Đặc trưng (Feature Extraction)

Để các thuật toán Học máy (như Logistic Regression, SVM) có thể xử lý, văn bản  $x_i$  phải được ánh xạ thành một vector số thực. Nhóm sử dụng phương pháp **TF-IDF (Term Frequency - Inverse Document Frequency)**.

#### 3.4.1 Term Frequency (TF)

Do lường tần suất xuất hiện của một từ  $t$  trong một văn bản  $d$ . Công thức chuẩn hóa TF để tránh sự thiên lệch đối với các văn bản dài là:

$$TF(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

Trong đó,  $f_{t,d}$  là số lần từ  $t$  xuất hiện trong văn bản  $d$ .

#### 3.4.2 Inverse Document Frequency (IDF)

Do lường mức độ quan trọng của từ  $t$  trong toàn bộ kho ngữ liệu  $D$  có kích thước  $N$ . Các từ xuất hiện ở quá nhiều văn bản sẽ bị giảm trọng số:

$$IDF(t, D) = \log \left( \frac{N}{|\{d \in D : t \in d\}|} \right)$$

Mẫu số  $|\{d \in D : t \in d\}|$  là số lượng văn bản chứa từ  $t$ . (Trong thực tế lập trình, một hằng số 1 thường được cộng vào mẫu số để tránh lỗi chia cho 0).

#### 3.4.3 Trọng số TF-IDF

Trọng số cuối cùng của từ  $t$  trong văn bản  $d$  là tích của TF và IDF:

$$TF\text{-}IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

Nhờ biến đổi này, mỗi văn bản  $x_i$  được biểu diễn thành một vector thưa (sparse vector)  $\mathbf{v}_i \in \mathbb{R}^V$ , với  $V$  là kích thước của từ điển (vocabulary size), tạo tiền đề lý tưởng cho các mô hình phân loại tuyến tính.

#### 3.4.4 Word2Vec

Word2Vec dựa trên giả thuyết phân phối (Distributional Hypothesis): các từ xuất hiện trong cùng ngữ cảnh thường có nghĩa gần nhau. Trong kiến trúc **Skip-gram**, mô hình dự đoán các từ ngữ cảnh dựa trên từ mục tiêu. Hàm mục tiêu tối đa hóa xác suất logarit:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log P(w_{t+j} | w_t) \quad (1)$$

#### 3.4.5 GloVe (Global Vectors for Word Representation)

Để hiện thực phương pháp Word2Vec, chúng ta cần train một model nông (shallow model), thường với 2 hidden layers để trả ra được các thông số mong muốn, trong bài toán này chúng ta sử dụng Glove là một file output từ một model được train từ trước (cụ thể: `wiki_giga_2024_100_MFT20_vectors_seed_2024_alpha_0.75_eta_0.05_combined`). GloVe tối ưu hóa hàm lỗi dựa trên ma trận hiệp xuất  $X$  (co-occurrence matrix) của toàn bộ kho ngữ liệu:

$$J = \sum_{i,j=1}^V f(X_{ij})(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2 \quad (2)$$

#### 3.4.6 So sánh Word Embeddings và TF-IDF

### 3.5 Mô hình học máy truyền thống

#### 3.5.1 Hồi quy Logistic (Logistic Regression)

Hồi quy Logistic là một mô hình phân loại xác suất tuyến tính, đặc biệt hiệu quả và phổ biến đối với bài toán phân loại văn bản nhị phân (Binary Classification) như Phân tích Sắc thái.

| Tiêu chí    | TF-IDF (Truyền thống)                                                | Word2Vec (Hiện đại)                                                                           |
|-------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Dạng Vector | Thưa (Sparse), số chiều khổng lồ bằng kích thước bộ từ vựng ( $V$ ). | Dày đặc (Dense), số chiều cố định, nhỏ gọn (ví dụ: 100D).                                     |
| Ngữ nghĩa   | Dựa vào tần suất đếm số cục bộ. Không hiểu mối quan hệ giữa các từ.  | Lưu trữ không gian hình học. Hiểu được ngữ nghĩa (Ví dụ: <i>king - man + woman ≈ queen</i> ). |
| Phạm vi học | Phụ thuộc hoàn toàn vào tập huấn luyện (Train data) hiện tại.        | Học bách khoa toàn thư từ kho dữ liệu khổng lồ (Wikipedia, Gigaword).                         |

### 3.5.1.a Hàm giả thuyết (Hypothesis Function)

Cho một vector đặc trưng đầu vào  $\mathbf{x} \in \mathbb{R}^V$  (trong trường hợp này là vector thưa TF-IDF với  $V$  là kích thước từ điển) và bộ trọng số  $\mathbf{w} \in \mathbb{R}^V$ , độ lệch (bias)  $b \in \mathbb{R}$ . Tổ hợp tuyến tính của đầu vào được biểu diễn bằng:

$$z = \mathbf{w}^T \mathbf{x} + b$$

Khác với Hồi quy Tuyến tính dự đoán một giá trị liên tục, Hồi quy Logistic cần ánh xạ giá trị  $z$  vào khoảng  $(0, 1)$  để biểu diễn xác suất hợp lệ. Việc này được thực hiện thông qua hàm kích hoạt Sigmoid (Logistic function):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Kết hợp lại, hàm giả thuyết dự đoán xác suất văn bản  $x$  mang nhãn Tích cực ( $y = 1$ ) được định nghĩa là:

$$\hat{y} = P(y = 1 | \mathbf{x}; \mathbf{w}, b) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

Dựa trên ngưỡng quyết định tiêu chuẩn, nếu  $\hat{y} \geq 0.5$ , mô hình sẽ phân loại văn bản vào lớp Tích cực; ngược lại, mô hình phân loại vào lớp Tiêu cực.

### 3.5.1.b Hàm mất mát (Binary Cross-Entropy Loss)

Việc sử dụng hàm chi phí bình phương tối thiểu (MSE) kết hợp với hàm Sigmoid sẽ tạo ra một hàm không lồi (non-convex), chứa nhiều cực tiểu cục bộ gây khó khăn cho việc tối ưu. Do đó, Hồi quy Logistic sử dụng hàm mất mát **Log-Loss** (Binary Cross-Entropy) dựa trên nguyên lý Ước lượng hợp lý cực đại (Maximum Likelihood Estimation).

Đối với tập dữ liệu gồm  $N$  mẫu, hàm mất mát cơ bản  $J(\mathbf{w}, b)$  được tính bằng:

$$J(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

## 3.5.2 Support Vector Machine - SVM

Trong bài toán phân loại văn bản với không gian đặc trưng TF-IDF có số chiều rất lớn, SVM với nhân tuyến tính (Linear Kernel) thường cho hiệu suất vượt trội. Mục tiêu của SVM là tìm ra một siêu phẳng (hyperplane) phân cách tối ưu giữa hai lớp dữ liệu Tích cực và Tiêu cực sao cho lề (margin) được cực đại hóa.

Phương trình của siêu phẳng trong không gian  $V$ -chiều được biểu diễn là:

$$\mathbf{w}^T \mathbf{x} + b = 0$$

Trong đó  $\mathbf{w}$  là véc-tơ pháp tuyến và  $b$  là độ lệch (bias).

Để xử lý các điểm dữ liệu nhiễu không thể phân tách tuyến tính hoàn toàn (Soft-margin SVM), các biến nới lỏng (slack variables)  $\xi_i \geq 0$  được đưa vào. Bài toán tối ưu hóa của SVM trở thành:

$$\min_{\mathbf{w}, b, \xi} \left( \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i \right)$$

Với điều kiện ràng buộc:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \forall i = 1, \dots, N$$

Siêu tham số  $C > 0$  quyết định sự đánh đổi giữa việc cực đại hóa lề (khi  $C$  nhỏ) và việc giảm thiểu sai số phân loại trên tập huấn luyện (khi  $C$  lớn). Nhóm sử dụng nhân tuyến tính vì bản thân dữ liệu văn bản đã thưa và có tính phân tách tuyến tính cao trong không gian số chiều lớn.



### 3.5.2.a Hàm mất mát Hinge (Hinge Loss)

Khác với Hồi quy Logistic sử dụng hàm Log-Loss nhằm tối ưu hóa xác suất, Support Vector Machine (SVM) tập trung vào việc cực đại hóa biên hình học (geometric margin) thông qua hàm mất mát **Hinge Loss**.

Để công thức toán học của Hinge Loss hoạt động, không gian nhãn thực tế  $y_i \in \{0, 1\}$  được biến đổi tạm thời về không gian  $y_i \in \{-1, 1\}$  (với -1 đại diện cho Tiêu cực và 1 đại diện cho Tích cực). Đối với một mẫu dữ liệu  $(\mathbf{x}_i, y_i)$  và hàm ra quyết định  $f(\mathbf{x}_i) = \mathbf{w}^T \mathbf{x}_i + b$ , hàm mất mát Hinge được định nghĩa là:

$$L(y_i, f(\mathbf{x}_i)) = \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$$

Bản chất của hàm Hinge Loss mang ý nghĩa hình học rất rõ ràng:

- **Khi**  $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$ : Điểm dữ liệu được phân loại đúng và nằm an toàn bên ngoài lề (margin). Khi đó,  $1 - y_i f(\mathbf{x}_i) \leq 0$ , và hàm mất mát trả về giá trị 0. Mô hình dừng cập nhật trọng số đối với các điểm này.
- **Khi**  $y_i(\mathbf{w}^T \mathbf{x}_i + b) < 1$ : Điểm dữ liệu có thể bị phân loại sai hoặc phân loại đúng nhưng nằm lấn vào vùng lề. Mô hình sẽ chịu một mức phạt tuyến tính tỷ lệ thuận với khoảng cách xâm phạm.

Về bản chất, các biến nới lỏng  $\xi_i$  trong bài toán tối ưu hóa Soft-margin SVM chính là biểu diễn trực tiếp của Hinge Loss:

$$\xi_i = \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$$

Việc tối thiểu hóa tổng Hinge Loss kết hợp với thành phần điều chuẩn  $\|\mathbf{w}\|^2$  giúp SVM không chỉ phân loại đúng tập huấn luyện mà còn đạt được khả năng tổng quát hóa (generalization) cao nhất đối với dữ liệu chưa nhìn thấy.

### 3.5.3 Bộ phân loại Naive Bayes Đa thức (Multinomial Naive Bayes)

Naive Bayes là một mô hình sinh thủy (generative model) dựa trên Định lý Bayes với giả định "thơ ngây" rằng các đặc trưng (các từ trong văn bản) là độc lập có điều kiện với nhau khi biết trước nhãn lớp  $y$ .

Xác suất hậu nghiệm để một văn bản  $d$  thuộc về lớp  $c \in \{0, 1\}$  được tính bằng:

$$P(c|d) = \frac{P(c)P(d|c)}{P(d)} \propto P(c) \prod_{t \in d} P(t|c)$$

Trong đó:

- $P(c)$  là xác suất tiên nghiệm của lớp  $c$ , được ước lượng bằng tỷ lệ văn bản thuộc lớp  $c$  trong tập huấn luyện.
- $P(t|c)$  là xác suất có điều kiện của từ  $t$  khi biết lớp  $c$ .

Để tránh hiện tượng xác suất bằng 0 khi một từ trong tập kiểm tra chưa từng xuất hiện trong tập huấn luyện, kỹ thuật làm mịn Laplace (Laplace Smoothing) được áp dụng:

$$P(t|c) = \frac{N_{tc} + \alpha}{N_c + \alpha|V|}$$

Với  $N_{tc}$  là tổng tần suất xuất hiện của từ  $t$  trong lớp  $c$ ,  $N_c$  là tổng số từ trong lớp  $c$ ,  $|V|$  là kích thước từ điển, và  $\alpha = 1$  là tham số làm mịn chuẩn.

## 3.6 Mô hình học sâu

### 3.6.1 Long Short-Term Memory (LSTM)

LSTM là một biến thể của Mạng nơ-ron hồi quy (RNN), được thiết kế để giải quyết vấn đề triệt tiêu đạo hàm (vanishing gradient) khi xử lý các chuỗi dữ liệu dài.

#### 3.6.1.a Tổng quan

##### 3.6.1.1 Vấn đề của RNN truyền thống

RNN chuẩn gặp khó khăn khi học các phụ thuộc dài hạn (long-range dependencies) do hiện tượng triệt tiêu đạo hàm. Khi lan truyền ngược qua thời gian (Backpropagation Through Time – BPTT) qua nhiều bước, gradient bị nhân liên tục với ma trận trọng số, dẫn đến hai vấn đề:

- Vanishing gradient: Gradient tiến về 0, mô hình không học được từ các bước thời gian xa.
- Exploding gradient: Gradient bùng nổ, làm mất ổn định quá trình huấn luyện.

### 3.6.1.2 Cơ chế bộ nhớ của LSTM

LSTM giải quyết vấn đề này bằng cách giới thiệu trạng thái ô (cell state)  $C_t$  – một “đường truyền thông tin” chạy dọc theo chuỗi với rất ít phép biến đổi, cho phép gradient lan truyền xa mà không bị suy hao. Thông tin được kiểm soát qua ba cổng học được.

| Cổng        | Ký hiệu | Chức năng                                        |
|-------------|---------|--------------------------------------------------|
| Forget gate | $f_t$   | Quyết định thông tin nào từ $C_{t-1}$ bị loại bỏ |
| Input gate  | $i_t$   | Quyết định thông tin mới nào được thêm vào       |
| Output gate | $o_t$   | Quyết định phần nào của cell state được xuất ra  |

### 3.6.1.b Cấu trúc toán học

LSTM kiểm soát dòng thông tin thông qua ba loại cổng chính: cổng quên (forget gate), cổng vào (input gate) và cổng ra (output gate). Tại mỗi bước thời gian  $t$ , tính toán được thực hiện như sau:

1. **Cổng quên ( $f_t$ ):** Quyết định thông tin nào từ trạng thái ô trước đó ( $C_{t-1}$ ) sẽ bị loại bỏ.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (3)$$

2. **Cổng vào ( $i_t$ ) và trạng thái ô dự tuyển ( $\tilde{C}_t$ ):** Quyết định thông tin mới nào sẽ được lưu vào trạng thái ô.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (4)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (5)$$

3. **Cập nhật trạng thái ô ( $C_t$ ):**

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (6)$$

4. **Cổng ra ( $o_t$ ) và trạng thái ẩn ( $h_t$ ):**

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (7)$$

$$h_t = o_t * \tanh(C_t) \quad (8)$$

Trong đó,  $\sigma$  là hàm sigmoid,  $*$  là phép nhân từng phần tử (Hadamard product),  $W$  và  $b$  là các ma trận trọng số và độ lệch.

### 3.6.2 BERT (Bidirectional Encoder Representations from Transformers)

BERT là mô hình ngôn ngữ dựa trên kiến trúc Transformer Encoder, nổi bật với khả năng mã hóa ngữ cảnh hai chiều (bidirectional).

#### 3.6.2.a Cơ chế Attention

- **Scaled Dot-Product Attention:** Với mỗi token đầu vào, ba vector được chiếu tuyến tính từ embedding: Query  $Q$ , Key  $K$  và Value  $V$ .

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (9)$$

Hệ số  $\sqrt{d_k}$  chuẩn hóa tránh giá trị dot-product quá lớn gây bão hòa softmax, với  $d_k$  là chiều của vector Key.

- **Multi-Head Attention:** Thay vì dùng một hàm attention duy nhất, BERT sử dụng  $k$  attention song song, mỗi đầu học một kiểu quan hệ ngữ nghĩa khác nhau:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (10)$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (11)$$

### 3.6.2.b Kiến trúc BERT

BERT-Base bao gồm 12 lớp Transformer Encoder xếp chồng, 12 attention heads mỗi lớp, hidden dimension 768 và khoảng 110 triệu tham số. Mỗi lớp Encoder gồm hai khối con:

1. **Multi-Head Self-Attention** – học quan hệ giữa các token (Phương trình (10)).
2. **Feed-Forward Network (FFN)** – biến đổi phi tuyến:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (12)$$

Mỗi khối con áp dụng Residual Connection và Layer Normalization:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x)) \quad (13)$$

### 3.6.2.c Biểu diễn đầu vào

BERT cộng ba loại embedding:

$$E_{\text{input}} = E_{\text{token}} + E_{\text{segment}} + E_{\text{position}} \quad (14)$$

| Embedding                 | Mô tả                                           |
|---------------------------|-------------------------------------------------|
| <b>Token Embedding</b>    | Biểu diễn từng từ/subword (WordPiece tokenizer) |
| <b>Segment Embedding</b>  | Phân biệt câu A và câu B (tác vụ cặp câu)       |
| <b>Position Embedding</b> | Mã hóa vị trí của token trong câu (learnable)   |

### 3.6.2.d Pre-training

BERT được huấn luyện trên lượng văn bản khổng lồ với hai mục tiêu:

1. **Masked Language Modeling (MLM)** 15% token đầu vào được che ngẫu nhiên bằng [MASK]. Mô hình học dự đoán token bị che dựa trên ngữ cảnh hai chiều:

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log P(x_i | \hat{x}) \quad (15)$$

trong đó  $\mathcal{M}$  là tập chỉ số các token bị che và  $\hat{x}$  là chuỗi đầu vào đã được mask.

2. **Next Sentence Prediction (NSP)** Mô hình phán đoán liệu câu B có thực sự là câu tiếp theo sau câu A hay không – giúp hiểu quan hệ liên câu, hữu ích cho các tác vụ như Question Answering và Natural Language Inference.

### 3.6.2.e Fine-tuning cho tác vụ cụ thể

Sau pre-training, BERT được fine-tune bằng cách thêm một lớp đầu ra nhỏ phù hợp với từng tác vụ downstream:

| Tác vụ                   | Đầu vào BERT                       | Đầu ra                    |
|--------------------------|------------------------------------|---------------------------|
| Phân loại văn bản        | [CLS] token embedding              | Lớp Softmax               |
| Nhận diện thực thể (NER) | Embedding mỗi token                | Softmax cho mỗi token     |
| Hỏi đáp                  | [CLS] câu hỏi [SEP] đoạn văn [SEP] | Vị trí bắt đầu & kết thúc |

## 4 Phương pháp Đánh giá Mô hình

Vì dữ liệu thực tế thường có sự mất cân bằng giữa các lớp, việc chỉ sử dụng Độ chính xác (Accuracy) là không đủ để phản ánh hiệu năng thực sự của mô hình. Do đó, nhóm sử dụng Ma trận nhầm lẫn (Confusion Matrix) cùng các chỉ số phái sinh.

## 4.1 Ma trận nhầm lẫn (Confusion Matrix)

Kết quả dự đoán được thống kê qua 4 đại lượng:

- **True Positive (TP):** Dự đoán đúng nhãn Tích cực.
- **True Negative (TN):** Dự đoán đúng nhãn Tiêu cực.
- **False Positive (FP):** Dự đoán sai nhãn Tiêu cực thành Tích cực (Lỗi loại I).
- **False Negative (FN):** Dự đoán sai nhãn Tích cực thành Tiêu cực (Lỗi loại II).

## 5 Hiện thực bài toán

### 5.1 Tiền xử lý dữ liệu

Đoạn mã hiện thực phần dọn dẹp và chuẩn hoá dữ liệu:

```
1 def clean_text(text):
2     """Clean text while preserving negative modifiers (Contraction Expansion)"""
3     if not isinstance(text, str):
4         return ""
5
6     text = re.sub(r'<.*?>', ' ', text)
7
8     # Contraction Expansion
9     text = re.sub(r'n't", " not", text)
10    text = re.sub(r'\re", " are", text)
11    text = re.sub(r'\s", " is", text)
12    text = re.sub(r'\d", " would", text)
13    text = re.sub(r'\ll", " will", text)
14    text = re.sub(r'\t", " not", text)
15    text = re.sub(r'\ve", " have", text)
16    text = re.sub(r'\m", " am", text)
17
18    # Keep letters, spaces, and safe apostrophes
19    text = re.sub(r"[^a-zA-Z\s']", '', text)
20    text = text.lower().strip()
21    text = re.sub(r'\s+', ' ', text)
22
23    return text
```

Đoạn mã trên sẽ chuẩn hoá tất cả các dữ liệu bao gồm các từ rút gọn, thay thế khoảng trống cũng như xác định và giữ các kí tự đặc biệt (nhưng an toàn) để toàn tập dữ liệu hoặc tập dữ liệu so với "từ điển" GloVe được đưa về cùng format chung (viết đầy đủ).

### 5.2 Xử lý Dữ liệu Thừa và Trích xuất Đặc trưng (Sparsity Handling & Feature Extraction)

Văn bản sau khi tiền xử lý được chuyển đổi thành các véc-tơ đặc trưng thông qua kỹ thuật **TF-IDF**. Đặc thù của dữ liệu văn bản là tạo ra một ma trận cực kỳ thưa thớt (sparse matrix) với hàng chục ngàn chiều. Để kiểm soát độ phức tạp và giảm thiểu nguy cơ quá khớp (overfitting), nhóm sử dụng hai chiến lược chính trong `TfidfVectorizer`:

- **Giới hạn không gian đặc trưng:** Thiết lập `max_features=5000` để chỉ giữ lại 5000 đặc trưng (n-gram) có tần suất xuất hiện cao nhất. Việc cắt tỉa này loại bỏ các từ vựng quá hiếm (nhiều), giúp giảm thiểu chiều dữ liệu và ép mô hình tập trung vào các đặc trưng cốt lõi.
- **Bổ sung ngữ cảnh cục bộ:** Thiết lập `ngram_range=(1, 2)` để trích xuất cả từ đơn (unigram) và cụm hai từ liên tiếp (bigram). Điều này giúp khắc phục phần nào nhược điểm mất trật tự từ của mô hình ngôn ngữ truyền thống, cho phép bắt được các cấu trúc phủ định (ví dụ: "not good").

**Đối với việc xử lý trích xuất đặc trưng bằng phương pháp Word2Vec:** cấu trúc tệp GloVe (Hoạt động như một Từ điển) - Khi tải tệp GloVe, cấu trúc thực chất là một bảng tra cứu (lookup table) dưới dạng **Dictionary**:

- **Key (Label):** Từ vựng thực tế (ví dụ: "amazing", "terrible").
- **Value (Vector):** Một mảng số thực có độ dài cố định  $N$  (ở đây  $N = 100$ ) đại diện cho tọa độ hình học của từ đó trong không gian ngữ nghĩa.

Khi tiền xử lý, nếu từ trong bài đánh giá có tồn tại trong các "Key" của GloVe, hệ thống sẽ ánh xạ từ đó thành "Value" (vector 100 chiều) để đưa vào mạng nơ-ron.

### 5.3 Cấu hình Mô hình và Điều chuẩn (Model Configuration & Regularization)

Để đảm bảo tính nhất quán và triệt tiêu nguy cơ rò rỉ dữ liệu (data leakage) ở bước vector hóa, toàn bộ các mô hình được triển khai thông qua cấu trúc Pipeline của `scikit-learn`.

**1. Hồi quy Logistic (Logistic Regression):** Mô hình được khởi tạo với cấu hình `LogisticRegression(max_iter=1000, C=1.0)`.

- **Điều chuẩn L2 (L2 Regularization):** Tham số nghịch đảo hệ số điều chuẩn được đặt ở mức mặc định  $C = 1.0$ . Thuật toán áp dụng một mức phạt tỷ lệ thuận với bình phương các trọng số, ngăn mô hình gán trọng số quá lớn cho bất kỳ từ vựng cá biệt nào, từ đó cải thiện khả năng tổng quát hóa trên tập kiểm tra.
- **Ngăn chặn Underfitting do chưa hội tụ:** Tham số `max_iter=1000` được thiết lập cao hơn mức mặc định. Không gian đặc trưng 5000 chiều đòi hỏi thuật toán tối ưu (solver) nhiều vòng lặp hơn để cập nhật gradient. Việc tăng số vòng lặp giúp tránh hiện tượng mô hình dừng sớm (early stopping) khi chưa đạt tới điểm cực tiểu toàn cục của hàm mất mát.

**2. Support Vector Machine (SVM):** Mô hình được hiện thực qua `SVC(kernel='linear', random_state=42)`. Theo lý thuyết học thống kê, khi số lượng đặc trưng ( $V = 5000$ ) tiệm cận hoặc vượt quá số lượng mẫu huấn luyện, dữ liệu có xu hướng phân tách tuyến tính rất rõ ràng. Do đó, nhân tuyến tính (`kernel='linear'`) được lựa chọn thay vì các nhân phi tuyến (RBF, Polynomial) nhằm tiết kiệm đáng kể chi phí tính toán đồng thời giảm thiểu rủi ro quá khớp.

- **Nhân tuyến tính (`kernel='linear'`):** Về mặt cơ chế, dữ liệu văn bản được biểu diễn qua TF-IDF tạo ra một không gian đặc trưng cực kỳ thưa thớt với kích thước  $V = 5000$  chiều. Trong những không gian siêu việt (hyper-dimensional) như vậy, các điểm dữ liệu hầu như luôn luôn có khả năng phân tách tuyến tính. Việc sử dụng các nhân phi tuyến phức tạp (như RBF hay Polynomial) không những ít mang lại lợi ích về độ chính xác mà còn làm tăng đột biến chi phí tính toán (độ phức tạp thời gian có thể tăng từ  $\mathcal{O}(N)$  lên đến  $\mathcal{O}(N^3)$ ) và gây ra rủi ro quá khớp (overfitting) nghiêm trọng. Do đó, nhân tuyến tính là lựa chọn tối ưu nhất về mặt lý thuyết học thống kê cho dữ liệu Text.
- **Cơ chế huấn luyện (Max-Margin Optimization):** Khác với Hồi quy Logistic tính toán sai số trên toàn bộ tập dữ liệu, trong quá trình thực thi hàm `.fit()`, thuật toán tối ưu của SVM chỉ tập trung vào một tập hợp nhỏ các điểm dữ liệu nằm sát ranh giới quyết định nhất (được gọi là Support Vectors). Mô hình xây dựng bộ trọng số  $\mathbf{w}$  sao cho lề (margin) hình học giữa hai lớp Tích cực và Tiêu cực là lớn nhất. Nhờ cơ chế bỏ qua các điểm dữ liệu nằm sâu ở vùng an toàn, SVM đặc biệt bền vững (robust) trước các điểm nhiễu (outliers) hoặc các đánh giá có ngôn từ mâu thuẫn.

**3. Multinomial Naive Bayes:** Được hiện thực bằng lớp `MultinomialNB()`. Cấu hình mặc định của mô hình này tích hợp sẵn kỹ thuật làm mịn Laplace ( $\alpha = 1.0$ ). Đây là biện pháp cốt lõi để xử lý các phần tử 0 trong ma trận thưa, đảm bảo xác suất tiên nghiệm không bị triệt tiêu hoàn toàn khi mô hình bắt gặp một từ vựng mới trong tập kiểm tra.

- **Làm mịn Laplace (`alpha=1.0`):** Đặc thù suy luận của Naive Bayes là thực hiện phép nhân liên tiếp các xác suất có điều kiện của từng từ vựng. Nếu một từ vựng  $t$  xuất hiện trong tập kiểm tra nhưng chưa từng xuất hiện ở lớp  $c$  trong tập huấn luyện, xác suất  $P(t|c) = 0$ . Phép nhân sẽ khiến toàn bộ xác suất của văn bản đó bị triệt tiêu về 0 (Zero-Frequency Problem). Tham số `alpha=1.0` được sử dụng để giải quyết triệt để rủi ro này bằng cách cộng thêm 1 vào tử số của mọi tần suất, đóng vai trò như một cơ chế điều chuẩn nhẹ giúp làm mềm phân phối xác suất.
- **Cơ chế huấn luyện (Maximum A Posteriori - MAP):** Điểm khác biệt mang tính bản chất của Naive Bayes so với hai mô hình trên là nó thuộc họ mô hình sinh thủy (Generative Model). Mô hình hoàn toàn không sử dụng các thuật toán tối ưu lặp (như Gradient Descent) để tìm cực tiểu hàm mất mát. Tại bước `.fit()`, quá trình huấn luyện thực chất là một chuỗi các phép đếm thống kê đại số tĩnh: đếm tần suất xuất hiện của nhãn để suy ra xác suất tiên nghiệm (Prior Probability), và đếm tần suất các từ trong không gian 5000 chiều để lập bảng xác suất có điều kiện (Likelihood). Nhờ cơ chế đếm trực tiếp này, Naive Bayes sở hữu tốc độ huấn luyện nhanh nhất trong toàn bộ cấu trúc Pipeline.

**Đoạn mã hiện thực trung tâm (Core Implementation):**

Listing 1: Mô đun xây dựng Pipeline và định tuyến mô hình (models.py)

```
1 def build_pipeline(model_name="logistic", max_features=5000):
2     tfidf = TfidfVectorizer(max_features=max_features, ngram_range=(1, 2))
3
4     if model_name == "logistic":
5         classifier = LogisticRegression(max_iter=1000, C=1.0)
6     elif model_name == "svm":
```

```

7 classifier = SVC(kernel='linear', random_state=42)
8 elif model_name == "naive_bayes":
9     classifier = MultinomialNB()
10
11 return Pipeline([
12     ('tfidf', tfidf),
13     ('classifier', classifier)
14 ])

```

**4. Long Short Term Memory (LSTM):** Mạng LSTM được thiết kế để nắm bắt ngữ cảnh phức tạp trong các bài đánh giá phim với các thông số cấu trúc sau:

- **Embedding Dimension:** 100 (Sử dụng pretrained matrix từ GloVe).
- **Hidden Dimension:** 256 để tạo không gian biểu diễn ẩn.

Đoạn mã hiện thực định nghĩa model:

```

1 class SentimentLSTM(nn.Module):
2     def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim, n_layers,
3         bidirectional, dropout, pretrained_embeddings=None):
4         super(SentimentLSTM, self).__init__()
5         self.embedding = nn.Embedding(vocab_size, embedding_dim)
6
7         if pretrained_embeddings is not None:
8             self.embedding.weight.data.copy_(pretrained_embeddings)
9
10        self.lstm = nn.LSTM(
11            embedding_dim,
12            hidden_dim,
13            num_layers=n_layers,
14            bidirectional=bidirectional,
15            dropout=dropout if n_layers > 1 else 0,
16            batch_first=True
17        )
18
19        self.dropout = nn.Dropout(dropout)
20        self.fc = nn.Linear(hidden_dim * 2 if bidirectional else hidden_dim, output_dim)

```

Đoạn mã hiện thực phần gọi model class với các thông số:

```

1 model = SentimentLSTM(
2     vocab_size=VOCAB_SIZE,
3     embedding_dim=100,
4     hidden_dim=256,
5     output_dim=1,
6     n_layers=2,
7     bidirectional=True,
8     dropout=0.5,
9     pretrained_embeddings=pretrained_embeddings
10 )
11 model = model.to(device)

```

**5. Bidirectional Encoder Representations from Transformers (BERT):** Nhóm thực hiện tinh chỉnh (Fine-tuning) mô hình bert-base-uncased để tối ưu cho bài toán phân tích cảm xúc.

- **Kiến trúc hệ thống (Architecture):** Sử dụng 12 lớp Transformer xếp chồng (Transformer Blocks) với 768 đơn vị ẩn và 12 đầu chú ý (Attention heads). Đầu ra của token đặc biệt [CLS] (đại diện cho ý nghĩa toàn câu) được đưa qua một lớp Tuyến tính (Linear Layer) để thực hiện nhiệm vụ phân loại nhị phân.
- **Tham số cấu hình và Tiền xử lý:**
  - **Max Length (128):** Mặc dù BERT hỗ trợ tối đa 512 tokens, nhóm ấn định ngưỡng 128 để đảm bảo tính toán ổn định trên GPU T4, tránh lỗi tràn bộ nhớ VRAM trong khi vẫn bao phủ đủ lượng từ khóa quan trọng từ các bài đánh giá phim.
  - **Batch Size (16):** Sử dụng kích thước lô nhỏ để giúp việc cập nhật trọng số diễn ra thường xuyên hơn, phù hợp với chiến lược tinh chỉnh trên tập dữ liệu lấy mẫu 5.000 bản ghi.
- **Điều chuẩn và Tối ưu hóa (Regularization & Optimization):**

- **AdamW Optimizer:** Sử dụng bộ tối ưu hóa Adam với kỹ thuật Weight Decay cải tiến. Cơ chế này giúp kiểm soát sự suy giảm trọng số tốt hơn Adam thông thường, ngăn chặn việc mô hình thay đổi quá mạnh các tri thức ngôn ngữ đã được huấn luyện sẵn (pre-trained).
- **Tốc độ học** ( $Learning Rate = 2 \times 10^{-5}$ ): Thiết lập tốc độ học cực thấp để quá trình hội tụ diễn ra ổn định, đảm bảo mô hình thích nghi với miền dữ liệu IMDB mà không làm mất đi khả năng tổng quát hóa ngôn ngữ cốt lõi.

Đoạn mã hiện thực nạp mô hình và cấu hình:

```

1 bert_tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
2 bert_model = BertForSequenceClassification.from_pretrained(
3     'bert-base-uncased',
4     num_labels=2
5 ).to(device)
6
7 optimizer = AdamW(bert_model.parameters(), lr=2e-5, eps=1e-8)

```

## 5.4 Tối ưu hóa Siêu tham số (Hyperparameter Tuning)

Việc sử dụng cấu hình mặc định có thể không khai thác tối đa năng lực của mô hình trên phân phối dữ liệu cụ thể. Để xác định bộ siêu tham số (hyperparameters) tối ưu một cách khách quan và có cơ sở toán học, nhóm áp dụng phương pháp **Grid Search** kết hợp **Kiểm chứng chéo (K-Fold Cross-Validation)**.

### 5.4.1 Thiết lập Không gian tìm kiếm (Search Space)

Quá trình tối ưu hóa được thực hiện đồng thời trên cả bộ trích xuất đặc trưng và mô hình phân loại (Logistic Regression). Không gian tìm kiếm  $\mathcal{H}$  được định nghĩa bằng tích Đề-các của các tập tham số:

- **Kích thước từ điển** ( $\text{max\_features}$ )  $\in \{3000, 5000, 7000\}$ : Đánh giá sự đánh đổi giữa việc giữ lại lượng thông tin chi tiết (7000) và việc loại bỏ nhiễu cục bộ (3000).
- **Cường độ điều chuẩn L2** ( $C$ )  $\in \{0.1, 1.0, 10.0\}$ : Kiểm tra giới hạn của hàm mất mát.  $C = 0.1$  đại diện cho một mô hình bị phạt nặng (ưu tiên tính tổng quát), trong khi  $C = 10.0$  đại diện cho mô hình linh hoạt (ưu tiên khớp chặt với dữ liệu huấn luyện).

Tổng cộng, có  $3 \times 3 = 9$  cấu hình đường ống (pipelines) được khởi tạo.

### 5.4.2 Kiểm chứng chéo K-Fold (K-Fold Cross-Validation)

Để tránh hiện tượng quá khớp trên tập kiểm tra (Test set leakage), nhóm không sử dụng tập Test trong quá trình dò tìm. Thay vào đó, tập huấn luyện được chia thành  $K = 5$  phần (folds) rời rạc.

Đối với mỗi cấu hình  $h \in \mathcal{H}$ : 1. Mô hình được huấn luyện trên  $K - 1$  phần dữ liệu. 2. Mô hình được đánh giá (validation) trên phần dữ liệu còn lại bằng chỉ số F1-Score. 3. Quá trình lặp lại  $K$  lần để mỗi phần dữ liệu đều được làm tập validation đúng một lần.

Hiệu năng thực sự của cấu hình  $h$  là trung bình cộng của  $K$  lần thử nghiệm:

$$\text{CV-Score}(h) = \frac{1}{K} \sum_{k=1}^K F1_k^{(h)}$$

### 5.4.3 Kết quả Tối ưu hóa

Sau quá trình duyệt toàn bộ không gian  $\mathcal{H}$ , thuật toán chọn ra bộ tham số  $h^* = \arg \max_{h \in \mathcal{H}} \text{CV-Score}(h)$ . Khung `GridSearchCV` của `scikit-learn` tự động giữ lại cấu hình  $h^*$  này và thực hiện một vòng huấn luyện cuối cùng trên toàn bộ dữ liệu Train. Mô hình tối ưu này sau đó mới được đưa ra suy diễn trên tập Test độc lập để ghi nhận kết quả cuối cùng cho báo cáo.



## 5.5 Chiến lược Huấn luyện và Đánh giá (Training & Evaluation Framework)

### 5.5.1 Đối với mô hình truyền thống:

- Quá trình chia tách dữ liệu được thực hiện qua `train_test_split` với tỷ lệ 80:20. Việc ấn định hạt giống ngẫu nhiên (`random_state=42`) trong tất cả các khâu (từ chia dữ liệu đến khởi tạo SVM) là yêu cầu bắt buộc nhằm đảm bảo tính tái lập (reproducibility) của thí nghiệm khoa học.
- Chu trình huấn luyện được thực thi khép kín qua phương thức `.fit()` của `Pipeline`. Lúc này, thuật toán thực hiện đồng thời hai tác vụ: học không gian từ vựng từ tập huấn luyện để định hình kích thước vector, và điều chỉnh tham số  $\mathbf{w}$  của các bộ phân loại tương ứng. Tại pha suy diễn (Inference), mô hình tự động bỏ qua các từ mới xuất hiện trong tập kiểm tra mà không gây ra lỗi tràn khung dữ liệu.

### 5.5.2 Đối với mô hình học sâu (LSTM):

- **Architecture:** Bidirectional LSTM (Cho phép mô hình tổng hợp luồng thông tin theo cả hai chiều: từ quá khứ đến tương lai và ngược lại).
- **Layers:** 2 lớp LSTM xếp chồng lên nhau nhằm trích xuất đặc trưng sâu hơn.
- **Regularization:** Áp dụng Dropout 0.5 giữa các lớp để triệt tiêu việc học vẹt.

### 5.5.3 Đối với mô hình học sâu (BERT):

- **Architecture:** Mô hình `bert-base-uncased` với 12 lớp Transformer Encoder xếp chồng, 12 Attention heads và 768 đơn vị ẩn ( $\approx 110$  triệu tham số). Đầu ra của token `[CLS]` được đưa qua một lớp Linear để thực hiện phân loại nhị phân.
- **Tokenization & Padding:** `BertTokenizer` với `WordPiece`, chuẩn hóa độ dài `max_length = 128 tokens` (truncation cho chuỗi dài, padding với `[PAD]` cho chuỗi ngắn).
- **Optimizer:** AdamW với  $1r = 2 \times 10^{-5}$ ,  $eps = 10^{-8}$ ,  $weight\_decay = 10^{-2}$ . Tốc độ học cực thấp giúp mô hình thích nghi với miền IMDB mà không làm mất kiến thức ngôn ngữ đã pre-train.
- **Batch Size:** 16 — kích thước lô nhỏ phù hợp với giới hạn VRAM của GPU T4 và tập dữ liệu lấy mẫu 5.000 mẫu.
- **Regularization:** Weight Decay ( $10^{-2}$ ) thông qua AdamW và Early Stopping (`patience = 3`) dựa trên Validation Loss để ngăn chặn overfitting trên tập huấn luyện nhỏ.

### 5.5.4 Sự khác biệt trong chiến thuật huấn luyện

Quá trình huấn luyện mạng học sâu có sự dịch chuyển lớn so với quy trình học máy truyền thống:

1. **Tỷ lệ chia dữ liệu (Data Split):** Thay vì chia 80/20 (Train/Test) như các mô hình học máy truyền thống, mô hình áp dụng chiến thuật cắt 3 phần: **70/15/15** (Train/Validation/Test).
2. **Giám sát bằng Validation Set:** Tập Validation hoạt động như một "máy đo nhịp tim" khách quan trong suốt các vòng lặp (Epochs).
3. **Early Stopping:** Khác với mô hình truyền thống tính toán biên giới ranh giới trong một lần, mạng nơ-ron cập nhật trọng số liên tục. Nhờ tập Validation, quy trình huấn luyện có thể chủ động dừng lại sớm (Early Stopping) ngay khi phát hiện Loss bắt đầu tăng trở lại, ngăn chặn hiệu quả hiện tượng quá khớp (Overfitting).

### 5.5.5 Điều chuẩn L2 (L2 Regularization)

Đặc thù của dữ liệu biểu diễn bằng TF-IDF là không gian đặc trưng có số chiều rất lớn (High-dimensionality). Để ngăn chặn mô hình học vẹt các đặc trưng nhiễu dẫn đến hiện tượng quá khớp (Overfitting), kỹ thuật điều chuẩn L2 (Ridge Regularization) được thêm vào hàm mất mát gốc:

$$J_{\text{reg}}(\mathbf{w}, b) = J(\mathbf{w}, b) + \frac{1}{2C} \sum_{j=1}^V w_j^2$$

Trong đó,  $\|\mathbf{w}\|_2^2 = \sum_{j=1}^V w_j^2$  là định mức L2 của vector trọng số.

Siêu tham số  $C > 0$  (nghịch đảo của hệ số phạt  $\lambda$ ) đóng vai trò kiểm soát mức độ điều chuẩn. Khi  $C$  càng nhỏ, mô hình càng bị phạt mạnh đối với các trọng số lớn, ép mô hình phải phân bổ trọng số đồng đều hơn cho các từ vựng. Trong phần thực nghiệm, các giá trị khác nhau của  $C$  sẽ được đánh giá thông qua chiến lược tìm kiếm lưới (Grid Search) để chọn ra cấu hình tối ưu nhất.

## 6 Kết quả và đánh giá

### 6.1 Phân tích đặc điểm dữ liệu (Exploratory Data Analysis - EDA)

Trước khi tiến hành huấn luyện, nhóm đã thực hiện phân tích đặc điểm của tập dữ liệu IMDB (trên toàn bộ mẫu) để đưa ra các thiết lập phù hợp cho mô hình.

#### 6.1.1 Phân phối nhãn dữ liệu (Class Distribution)

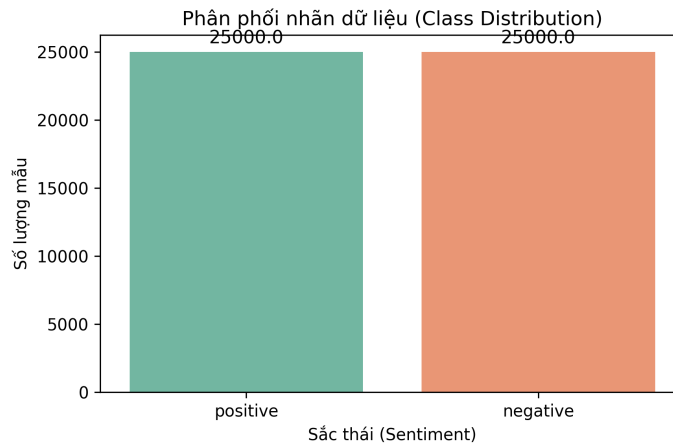
Tập dữ liệu IMDB gồm tổng cộng  $N = 50,000$  mẫu. Dựa trên Bảng 1 và Hình ??, có thể thấy tập dữ liệu cho thấy sự cân bằng lý tưởng giữa hai lớp Positive và Negative:

Table 1: Thống kê số lượng mẫu theo nhãn cảm xúc.

| Nhãn                          | Số lượng mẫu  | Tỷ lệ       |
|-------------------------------|---------------|-------------|
| Tích cực (Positive, $y = 1$ ) | 25,000        | 50%         |
| Tiêu cực (Negative, $y = 0$ ) | 25,000        | 50%         |
| <b>Tổng cộng</b>              | <b>50,000</b> | <b>100%</b> |

Tỷ lệ xác suất tiên nghiệm  $P(y = 1) = P(y = 0) = 0.5$  cho thấy tập dữ liệu hoàn toàn cân bằng. Điều này mang lại hai lợi ích quan trọng:

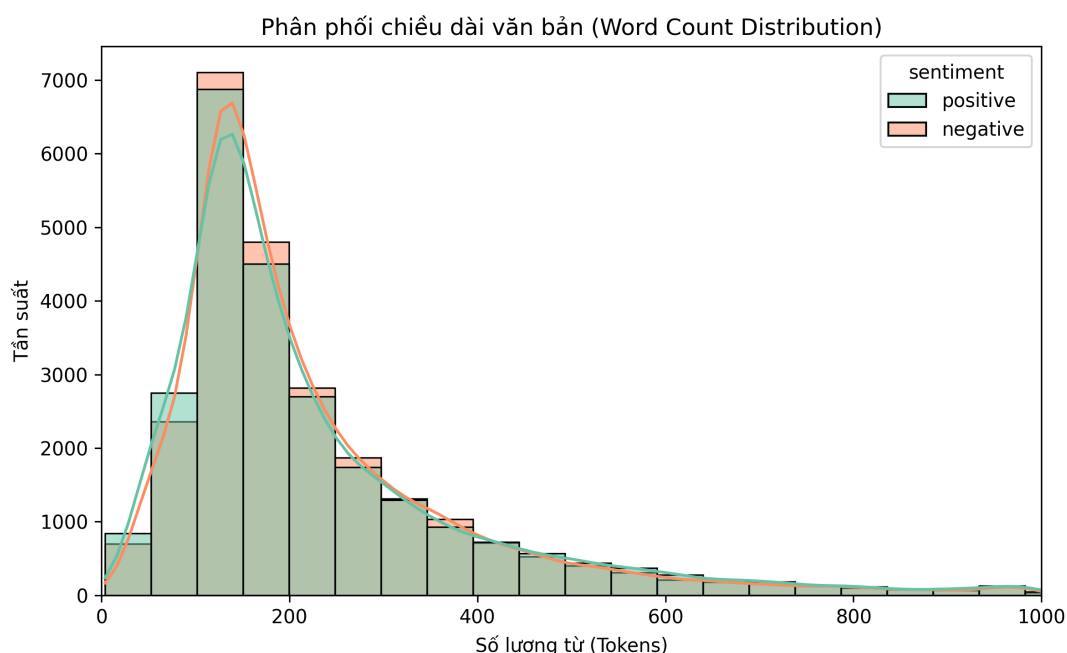
- **Về mặt huấn luyện:** Giúp mô hình tránh được hiện tượng thiên kiến (bias) đối với một lớp cụ thể. Nhóm không cần áp dụng các kỹ thuật xử lý mất cân bằng như Class Weights hay Oversampling.
- **Về mặt đánh giá:** Cho phép sử dụng các độ đo Accuracy và F1-Score làm chỉ số đánh giá chính mà không lo ngại kết quả bị phản ánh sai lệch do sự áp đảo của một lớp dữ liệu.



#### 6.1.2 Phân phối độ dài văn bản (Word Count Distribution)

Chiều dài  $L_i$  của văn bản  $x_i$  được tính bằng số lượng từ (tokens) sau bước tiền xử lý. Các thống kê mô tả của tập dữ liệu được tổng hợp như sau:

| Thống kê                   | Positive            | Negative            |
|----------------------------|---------------------|---------------------|
| Trung bình ( $\mu$ )       | $\approx 230$ từ    | $\approx 230$ từ    |
| Trung vị (median)          | $\approx 170$ từ    | $\approx 160$ từ    |
| Độ lệch chuẩn ( $\sigma$ ) | $\approx 170$ từ    | $\approx 175$ từ    |
| Giá trị nhỏ nhất           | $< 10$ từ           | $< 10$ từ           |
| Giá trị lớn nhất           | $\approx 1,000+$ từ | $\approx 1,000+$ từ |



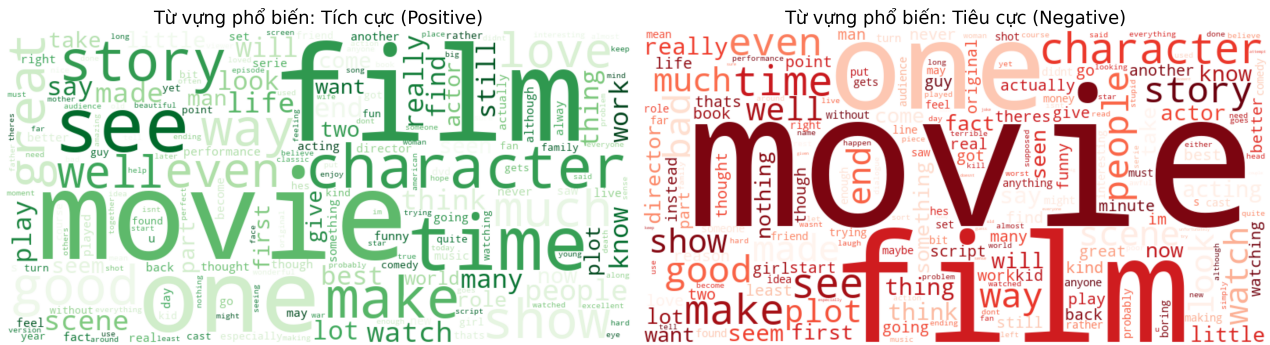
Phân phối chiều dài của cả hai lớp đều lệch phải (right-skewed), với đỉnh nằm trong khoảng 100–150 từ và đuôi phân phối kéo dài đến hơn 1,000 từ. Hai đường KDE của lớp Tích cực và Tiêu cực gần như trùng khít nhau, cho thấy **chiều dài văn bản không phải là đặc trưng phân biệt** giữa hai lớp. Điều này củng cố quyết định sử dụng các kỹ thuật trích xuất đặc trưng ngữ nghĩa (TF-IDF, GloVe) thay vì thông tin chiều dài thuần túy.

Độ lệch chuẩn lớn ( $\sigma \approx 170$ ) đòi hỏi phải áp dụng chiến lược truncation (cắt ngắn) và padding (đệm) khi đưa dữ liệu vào các mô hình học sâu để đảm bảo đầu vào có kích thước đồng nhất. Sự phân tán dữ liệu lớn dẫn đến hai quyết định kỹ thuật quan trọng trong Pipeline:

- **Xác định độ dài giới hạn:** Do đặc thù kiến trúc và tài nguyên phần cứng, hai mô hình học sâu sử dụng ngưỡng khác nhau:
  - **LSTM:** `max_length = 256` — ngưỡng này cao hơn giá trị trung bình ( $\mu \approx 230$ ) và trung vị, đảm bảo bao phủ hơn 75% các mẫu mà không làm mất các từ khoá cảm xúc quan trọng ở cuối văn bản.
  - **BERT:** `max_length = 128` — rút ngắn so với giới hạn tối đa 512 tokens của BERT do ràng buộc bộ nhớ VRAM của GPU T4, đồng thời vẫn đủ để bao phủ phần lớn nội dung cảm xúc cốt lõi.
- **Kỹ thuật xử lý:** Áp dụng Truncation cho các review quá dài và Padding cho các review ngắn để tạo ra các tensor có kích thước đồng nhất, giúp tối ưu hoá việc tính toán song song trên GPU.

### 6.1.3 Tần suất xuất hiện của từ (Word Frequency & Word Cloud)

Để phân tích ngữ nghĩa tổng quan, nhóm xây dựng *Word Cloud* riêng cho từng lớp cảm xúc sau bước loại bỏ stop words.



Một số nhận xét quan trọng được rút ra:

- **Từ vựng trung tính chung:** Các từ như *movie, film, character, story, time, one* xuất hiện nổi bật ở cả hai lớp với tần suất cao, phản ánh chủ đề miền (domain) của tập dữ liệu điện ảnh hơn là tín hiệu cảm xúc. Đây chính là lý do TF-IDF (thông qua thành phần IDF) có giá trị làm **giảm trọng số** các từ xuất hiện đồng đều ở mọi văn bản.
- **Từ phân cực Tích cực:** Các từ *see, good, love, best, enjoy, beautiful, well, classic* xuất hiện nổi trội hơn ở lớp Positive, mang tín hiệu đánh giá thuận chiều.
- **Từ phân cực Tiêu cực:** Các từ *nothing, bad, boring, plot* (theo nghĩa chỉ trích), *end* (ngụ ý kết thúc tệ), *even* (mang hàm ý phủ định như “not even good”) xuất hiện nổi trội hơn ở lớp Negative.
- **Kết luận phương pháp:** Sự tồn tại của các từ phân cực rõ ràng xác nhận rằng bài toán có **tính phân tách tuyến tính cao** trong không gian TF-IDF, ủng hộ lựa chọn SVM với nhân tuyến tính và Logistic Regression làm baseline.

## 6.2 Mô tả Pipeline và các tham số đã thử nghiệm

### 6.2.1 Tổng quan kiến trúc Pipeline

Toàn bộ hệ thống được thiết kế theo ba nhánh pipeline độc lập, tương ứng với các hướng tiếp cận từ truyền thống đến hiện đại. Hình 1 mô tả luồng xử lý tổng thể.

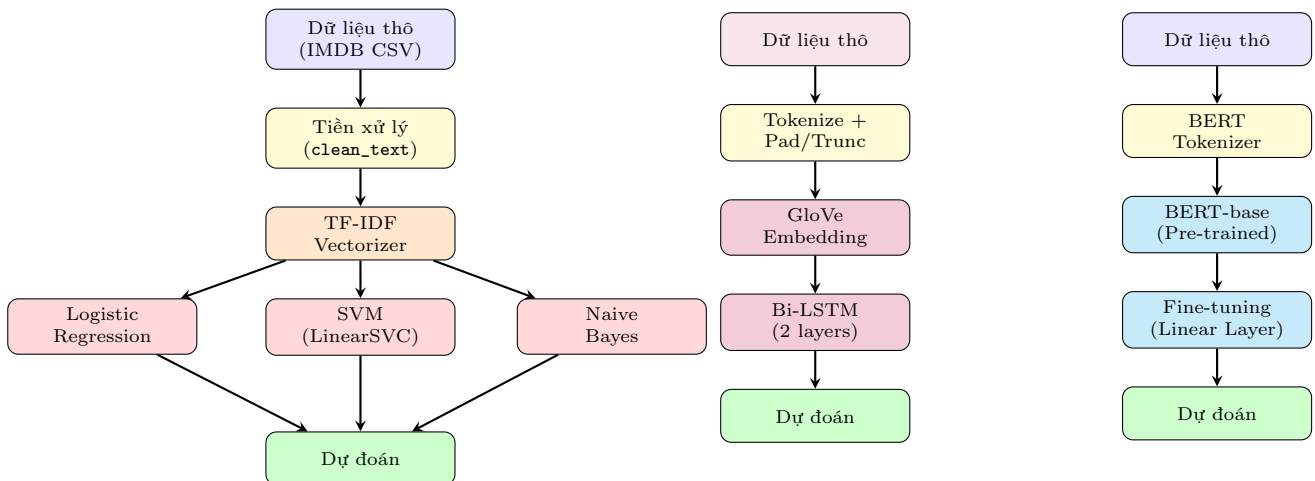


Figure 1: Tổng quan ba nhánh Pipeline: Truyền thống, Học sâu LSTM và Transformer BERT.

### 6.2.2 Bảng tổng hợp tham số cấu hình (Hyperparameter Configuration)

Bảng tổng hợp toàn bộ các siêu tham số được sử dụng trong từng mô hình, bao gồm cả giá trị mặc định (baseline) và không gian tìm kiếm (Grid Search).

| Mô hình             | Tham số        | Giá trị thử nghiệm        | Giá trị tối ưu               |
|---------------------|----------------|---------------------------|------------------------------|
| TF-IDF              | max_features   | {3000, 5000, 7000}        | 7000                         |
|                     | ngram_range    | (1, 1), (1, 2)            | (1, 2)                       |
|                     | sublinear_tf   | True                      | True                         |
| Logistic Regression | C              | {0.1, 1.0, 10.0}          | 1.0                          |
|                     | max_iter       | 1000                      | 1000                         |
|                     | penalty        | L2                        | L2                           |
| SVM (LinearSVC)     | C              | {0.1, 1.0, 10.0}          | 0.1                          |
|                     | kernel         | linear                    | linear                       |
|                     | random_state   | 42                        | 42                           |
| Naive Bayes         | alpha          | 1.0 (Laplace)             | 1.0                          |
|                     | fit_prior      | True                      | True                         |
| LSTM                | embedding_dim  | 100 (GloVe pretrained)    | 100                          |
|                     | hidden_dim     | {128, 256}                | 256                          |
|                     | n_layers       | 2                         | 2                            |
|                     | bidirectional  | True                      | True                         |
|                     | dropout        | 0.5                       | 0.5                          |
|                     | max_length     | 256                       | 256                          |
|                     | batch_size     | 64                        | 64                           |
| Huấn luyện LSTM     | optimizer      | Adam                      | Adam                         |
|                     | learning_rate  | 1e-3                      | 1e-3                         |
|                     | max_epochs     | 10                        | –                            |
|                     | early_stopping | patience = 3              | dừng tại epoch $\approx 2-3$ |
| Data Split          | Truyền thống   | Train/Test = 80/20        | –                            |
|                     | Học sâu        | Train/Val/Test = 70/15/15 | –                            |
|                     | random_state   | 42                        | 42                           |
| Cross-Validation    | K              | 5                         | –                            |
|                     | Metric         | F1-Score (macro)          | –                            |
| BERT                | model_name     | bert-base-uncased         | bert-base-uncased            |
|                     | max_length     | 128 (GPU limit)           | 128                          |
|                     | optimizer      | AdamW                     | AdamW                        |
|                     | learning_rate  | $2 \times 10^{-5}$        | $2 \times 10^{-5}$           |
|                     | batch_size     | 16                        | 16                           |

### 6.2.3 Tóm tắt thực nghiệm Grid Search

Quá trình Grid Search được thực hiện trên không gian tích Đề-các của  $\text{max\_features} \times C$ , tạo ra tổng cộng  $2 \times 3 = 6$  cấu hình pipeline cho mỗi mô hình truyền thống. Bảng 2 trình bày CV-Score (F1-Score trung bình qua 5 folds) của tất cả 6 cấu hình cho mô hình SVM — mô hình đạt kết quả tốt nhất sau tuning.

| max_features                                                     | C    | CV-Score (F1 macro) |
|------------------------------------------------------------------|------|---------------------|
| 5000                                                             | 0.1  | 0.891289            |
| 7000                                                             | 0.1  | <b>0.894250</b>     |
| 5000                                                             | 1.0  | 0.885524            |
| 7000                                                             | 1.0  | 0.888906            |
| 5000                                                             | 10.0 | 0.866745            |
| 7000                                                             | 10.0 | 0.863025            |
| <b>Cấu hình tối ưu <math>h^*</math> max_features=7000, C=0.1</b> |      |                     |

Table 2: Kết quả Grid Search 5-Fold CV cho mô hình SVM (LinearSVC). Giá trị CV-Score là F1-Score macro trung bình qua 5 folds.

Cấu hình tối ưu  $h^* = \{\text{max\_features} = 7000, C = 0.1\}$  được chọn với CV-Score = 0.894250. Nhận xét:

- **max\_features = 7000** cho kết quả tốt hơn 5000. Điều này cho thấy với tập dữ liệu lớn như IMDB, việc mở rộng không gian đặc trưng giúp mô hình giữ lại được nhiều cụm từ đôi (n-grams) có giá trị biểu cảm cao, hỗ trợ tốt hơn cho việc phân biệt sắc thái cảm xúc phức tạp.
- **C = 0.1** mang lại hiệu quả cao nhất. Điều này có thể giải thích qua lý thuyết học thống kê: trong một không gian đặc trưng siêu việt (7000 chiều), dữ liệu rất dễ bị phân tách quá mức dẫn đến quá khớp (overfitting). Việc sử dụng  $C$  nhỏ giúp "làm mềm" ranh giới quyết định (soft margin), ưu tiên khả năng tổng quát hóa trên dữ liệu chưa biết hơn là cố gắng khớp hoàn toàn dữ liệu huấn luyện. Ngược lại, khi tăng  $C$  lên 10.0, F1-score giảm mạnh xuống còn khoảng 0.86, minh chứng cho hiện tượng mô hình đang tập trung quá mức vào các điểm nhiễu (outliers).

### 6.3 Các chỉ số đánh giá (Precision, Recall và F1-Score)

Dựa trên Ma trận nhầm lẫn, các chỉ số đánh giá cốt lõi được định nghĩa như sau:

**1. Precision (Độ chuẩn xác):** Tỷ lệ các mẫu thực sự Tích cực trong tổng số các mẫu được dự đoán là Tích cực.

$$\text{Precision} = \frac{TP}{TP + FP}$$

**2. Recall (Độ bao phủ):** Tỷ lệ các mẫu Tích cực được mô hình nhận diện đúng trên tổng số các mẫu Tích cực thực tế.

$$\text{Recall} = \frac{TP}{TP + FN}$$

**3. F1-Score:** Là trung bình điều hòa (Harmonic Mean) của Precision và Recall, giúp đánh giá cân bằng trong trường hợp tập dữ liệu bị lệch.

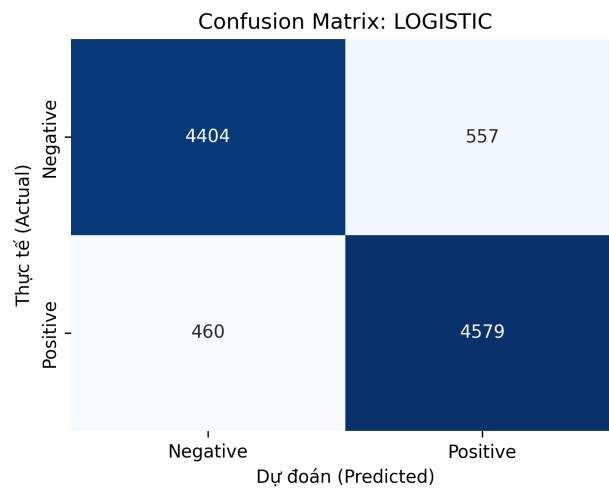
$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Trong bài toán thực tế, việc so sánh hiệu năng giữa các mô hình như SVM, Logistic Regression và Naive Bayes sẽ được quyết định chủ yếu dựa trên F1-Score.

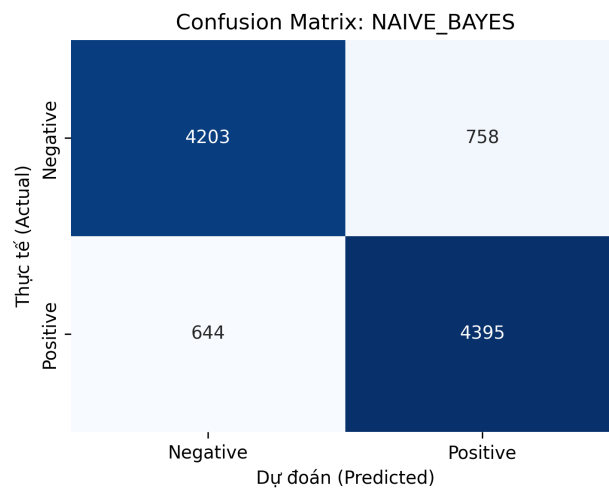


## 6.4 Pipeline truyền thống:

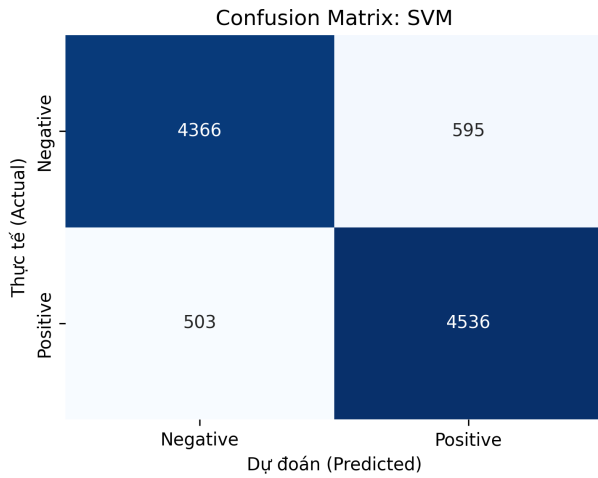
### 6.4.1 Logistic Regression:



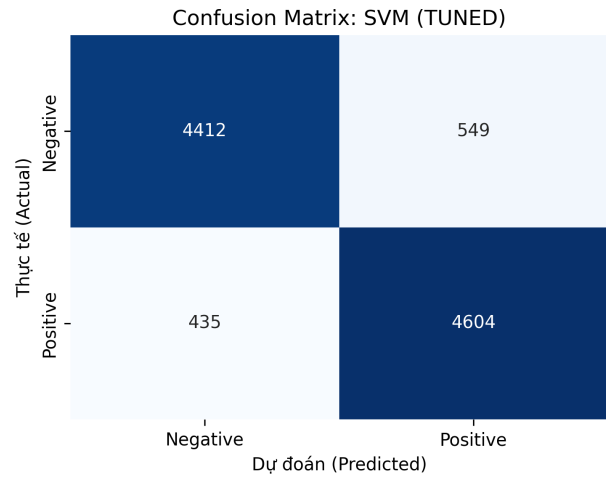
### 6.4.2 Naives Bayes:



### 6.4.3 SVM:

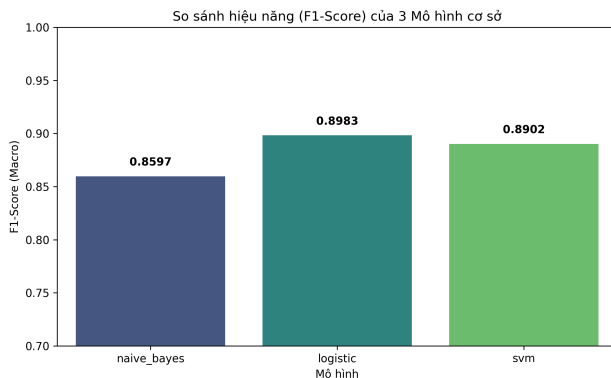


(a) SVM baseline

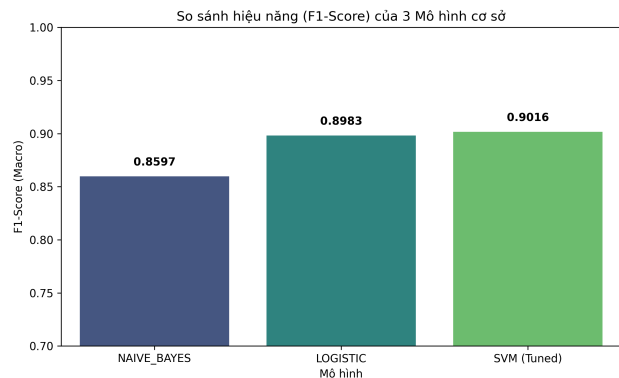


(b) SVM with GridSearch

### 6.4.4 So sánh giữa các model truyền thống:



(a) Baseline

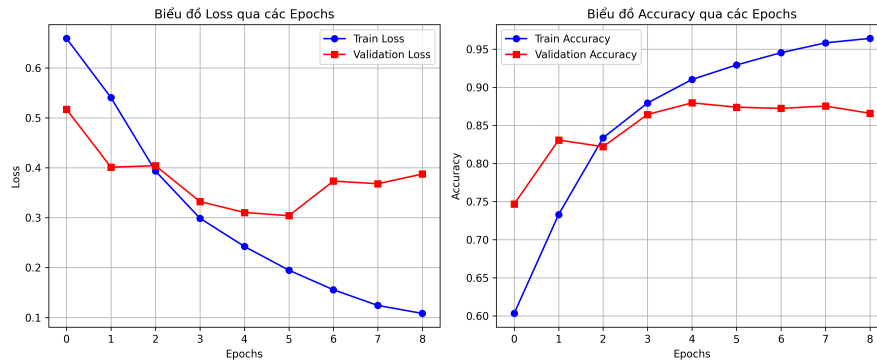


(b) After tuned

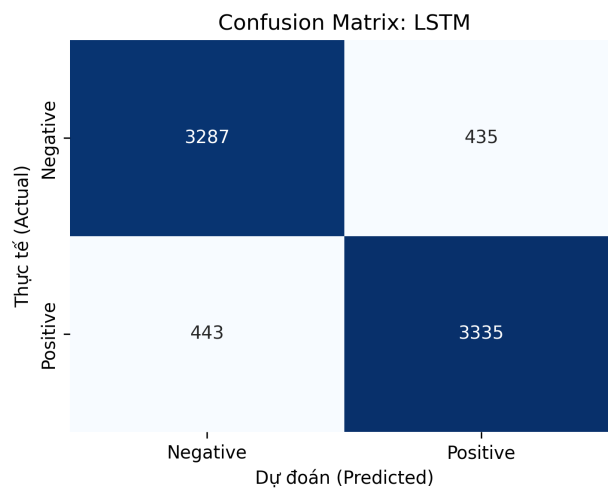
Có thể thấy, tương tự như việc độ chính xác của model SVM tăng sau khi thực hiện hyper parameter tuning bằng GridSearch, so sánh độ chính xác của các model truyền thống trước và sau khi hiệu chỉnh cũng có sự thay đổi về thứ tự, trong đó SVM trước khi tune là model có accuracy thấp hơn các model khác lại trở thành model có accuracy cao nhất.

## 6.5 Deeplearning Pipeline:

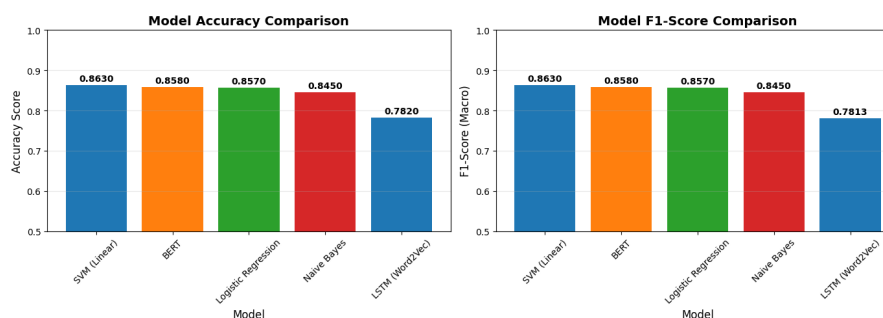
Model LSTM được hiện thực để sử dụng chiến thuật early stopping để ngăn chặn overfit, trong quá trình training cho model, train loss và validation loss được theo dõi để thực thi chiến thuật và có biểu hiện sau:



Có thể thấy, train loss và validation loss có sự phân kỳ bắt đầu từ khoảng epoch thứ 2. Confusion matrix của model LSTM với early stopping:



## 6.6 So sánh giữa pipeline truyền thống, deeplearning và model hiện đại - BERT:



Trong điều kiện thực nghiệm bị giới hạn tài nguyên cho thấy một hiện tượng đáng chú ý: mô hình học máy truyền thống (LinearSVC) đạt độ chính xác cao nhất, vượt qua các kiến trúc học sâu phức tạp như LSTM và Transformer (BERT). Hiện tượng này phản ánh đúng bản chất cơ học của các mô hình và có thể được giải thích qua ba nguyên nhân cốt lõi:

### 6.6.1 Ràng buộc về Kích thước Dữ liệu (Data Starvation)

- **Mô hình truyền thống (SVM):** Bản chất toán học của SVM là thiết lập siêu phẳng (hyperplane) phân cách dựa trên các vector hỗ trợ (support vectors). Nhờ cơ chế này, thuật toán hoạt động cực kỳ ổn định và chống nhiễu tốt ngay cả khi tập dữ liệu được lấy mẫu với kích thước nhỏ (5.000 mẫu).

- **Mô hình học sâu (LSTM & BERT):** Các kiến trúc này sở hữu không gian tham số khổng lồ (ví dụ: BERT-base có xấp xỉ 110 triệu tham số). Việc tinh chỉnh (fine-tuning) một mạng nơ-ron đồ sộ trên một tập mẫu bị giới hạn khiến mô hình rơi vào trạng thái "đói dữ liệu". Mô hình không có đủ sự đa dạng thông tin để tổng quát hóa các trọng số, dẫn đến hiện tượng hội tụ kém hoặc quá khớp (overfitting).

#### 6.6.2 Bản chất của Bài toán Phân tích Cảm xúc

- **Sức mạnh của TF-IDF:** Bài toán phân loại cảm xúc (Sentiment Analysis) phụ thuộc rất lớn vào các từ khóa mang tính phân cực (polarity keywords). Kết hợp TF-IDF đóng vai trò như một bộ lọc nhiễu xuất sắc, vứt bỏ các từ vô nghĩa và phóng to trọng số của các từ khóa mang tính quyết định, giúp SVM dễ dàng tính toán ranh giới.
- **Hạn chế của LSTM:** LSTM trích xuất ngữ cảnh dựa trên sự phụ thuộc chuỗi thời gian. Tuy nhiên, với các bài đánh giá dài, khoảng cách giữa các từ khóa phân cực có thể rất xa. Trên một tập dữ liệu nhỏ, LSTM gặp khó khăn trong việc duy trì bộ nhớ dài hạn (long-term dependency), dẫn đến nhiễu thông tin và đạt hiệu suất thấp nhất trong thực nghiệm.

#### 6.6.3 Chênh lệch trong Không gian Tối ưu hóa (Hyperparameter Tuning)

- **Tối ưu hóa triệt để (Exhaustive Search):** Mô hình LinearSVC đã được áp dụng thuật toán GridSearchCV để quét cạn không gian tham số. Việc tìm ra tổ hợp hoàn hảo nhất cho hằng số phạt  $C$  và `max_features` đã giúp mô hình phát huy 100% công lực.
- **Giới hạn tài nguyên phần cứng:** Ngược lại, do rào cản về bộ nhớ VRAM và thời gian huấn luyện, mô hình BERT và LSTM chỉ được huấn luyện với cấu hình cơ sở trong số lượng Epoch ngắn (2-3 vòng). Các mô hình này chưa được tối ưu hóa siêu tham số (hyperparameter tuning).

**Kết luận:** Kết quả thực nghiệm minh chứng rõ nét cho nguyên lý *Ockham's Razor* trong Học máy. Đối với các tập dữ liệu có kích thước giới hạn và bài toán mang tính phân cực từ vựng cao, một pipeline trích xuất đặc trưng thống kê (TF-IDF) kết hợp mô hình tuyến tính tối ưu hóa tốt (SVM) là sự lựa chọn hiệu quả, vượt trội và tiết kiệm tài nguyên nhất.

## Nguồn tham khảo

- [1] Python Documentation, “Our Documentation” [Online]. Available: <https://www.python.org/doc/>.
- [2] Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.
- [3] Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735-1780.
- [4] Pennington, J., Socher, R., & Manning, C. D. (2014). *GloVe: Global Vectors for Word Representation*. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), 1532-1543.
- [5] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv preprint arXiv:1810.04805.
- [6] Paszke, A. et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems 32.